



JavaScript

講義資料 - 後期

今回の内容 - 教科書準拠

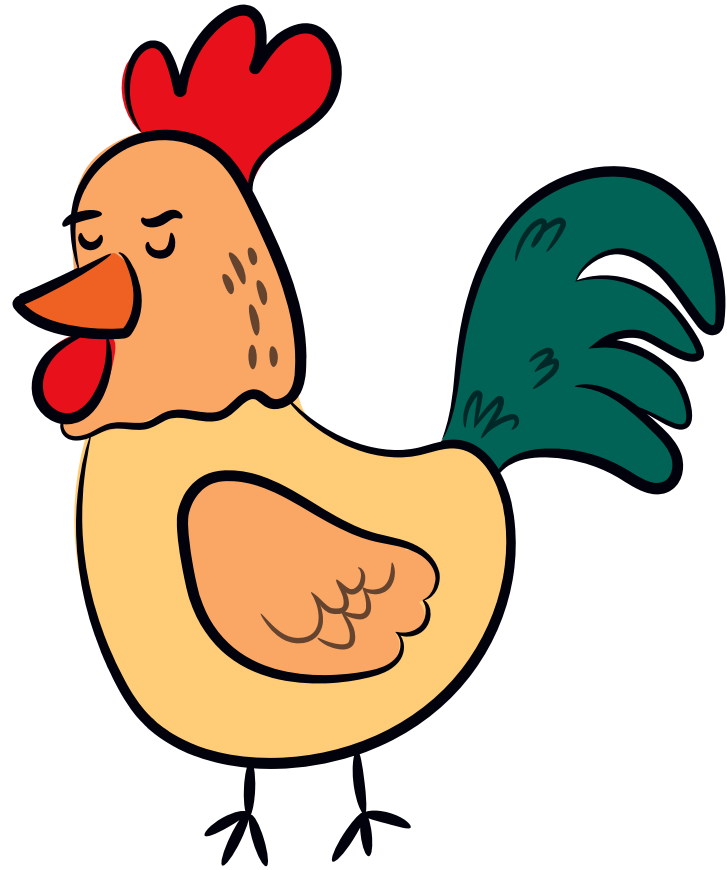
- 1.1-1 クラスとは？
- 2.1-2 クラスの使い方
- 3.1-3 静的なプロパティ
- 4.1-4 クラス継承
- 5.1-5 コードを実装してみよう

1-1 クラスとは？

クラスの考え方

複数の変数と関数(メソッド)で構成されるプログラムのかたまり

クラスに属する関数をメソッド、変数をフィールドと表現する。
実体である「オブジェクト」に対し、「クラス」は設計書に近いものとも言われる。



- 名前は？
- 鳴き声は？
- どう動く？

…動物として共通して持っている特徴である。

しかし、それぞれの動物によって違いがある。

∴「名前」、「鳴き声」、「動き方」という定義(設計書)を用意しておき、具体的なデータは必要な時に後で割り当てる。

1-2 クラスの使い方

クラスの宣言方法

通常のカラス宣言

```
class Sample {  
    // ここにカラスの宣言  
}
```

カラス式型の宣言方法

```
const Sample2 = class ClassName {  
    // ここに宣言  
}
```

講義内では、通常のカラス宣言のみを使用する。
「late/chapter1/sample2.html」を参照。

1-2 クラスの使い方

プロパティを定義する

フィールドとメソッドの定義方法

```
class Sample3 {  
    varSample;  
    methodSample() {  
        // メソッドの処理  
    }  
}
```

コンストラクタの使い方

「コンストラクタ」とは、実体を宣言した時に実行される特別なメソッド。
初期化や初期設定など、何かしら最初にやっておく必要がある場合に実行させる。

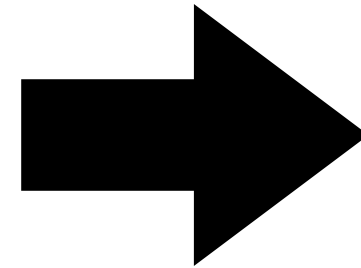
```
class Sample3 {  
    constructor(value1) {  
        // ここにコンストラクタの初期化処理  
    }  
}
```

クラスの仕組みを理解しよう

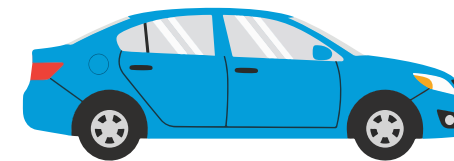
■車クラス(設計図)

- ー走るメソッド
- ー値引きメソッド
- ー名前フィールド
- ー重さフィールド

プロパティ



具体的なデータとして
実体化
(インスタンス)



名前: ガソリン車

重さ: 300kg

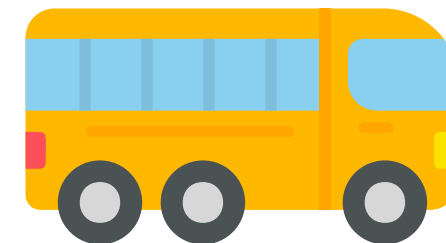
走るメソッド: 形と重さに依存



名前: トラック

重さ: 500kg

走るメソッド: 形と重さに依存



名前: バス

重さ: 1,000kg

走るメソッド: 形と重さに依存

フィールドとメソッドの定義方法

変数名 = new クラス名(コンストラクタへの引数);

コンストラクタがない場合と、コンストラクタに引数がない場合も空欄で()はつける。

【記述例】

```
const classIns = new Sample();
```

1-2 クラスの使い方

publicとprivateなプロパティ

公開による宣言(public: パブリック)とは？

実体から直接アクセス可能なプロパティのこと。

通常の変数・関数のように宣言した場合にデフォルトで適用される。

private含め比較的最近できた仕様のため、プロジェクトによっては使い分けがされていない可能性あり(ES2022)。

非公開による宣言(private: プライベート)とは？

実体からの直接アクセスができないプロパティのこと。

具体的に使う場合は、クラス内の他のメソッドなどから呼ばれた時。

外部から直接使う必要のない場合にはこちらで宣言する。

変数をprivateで、メソッドをpublic管理するのが「オブジェクト指向」では一般的な使い方。

↓ privateでは頭に「#」をつける

```
#fieldName;
```

```
#privateMethod() {}
```

1-2 クラスの使い方

例題

「sample2.html」を参考に「example2.html」をやってみよう。

記述内容はコメントで指示しているので、それにしたがって記入する。

1-3 静的なプロパティ

静的とは？

動的な使い方との違いを理解する

静的な使い方の場合は、クラスを実体化せずに直接扱うことができる。

※new の実体化文が不要。

一方で実体を通さず使うためクラス内で共通となっており、実体別の設定はできない。

✓ サンプルで言うと、犬とニワトリのような分け方が出来なくなる。

詳細は「sample3.html」を見てみよう。

【記述方法】

宣言の前に「static」を付ける

```
static name;
```

```
static methodName() {}
```

呼び出す時は、クラス名でアクセスできる。

```
ClassName.name;
```

```
ClassName.methodName();
```

1-3 静的なプロパティ

静的プロパティと実体

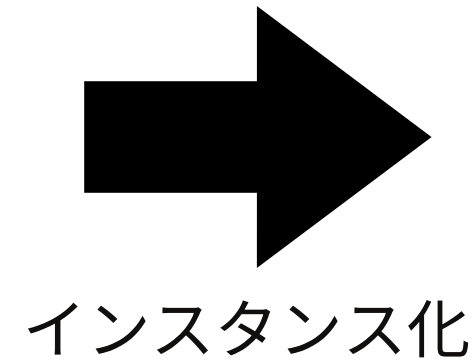
■日本車クラス(設計図)

■静的プロパティ(実体)

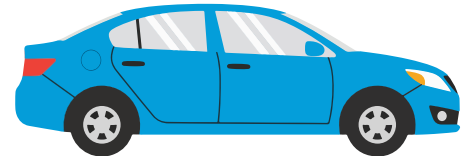
- ーstatic 走るメソッド
- ーstatic 生産国

■プロパティ(設計図のみ)

- ー社名表示メソッド
- ー名前フィールド



- ーメソッド: T社製ですと表示
- ー名前: T社
- ×走るメソッドは直接呼べない
- ×生産国フィールドは直接呼べない



- ーメソッド: MS社製ですと表示
- ー名前: MS社
- ×走るメソッドは直接呼べない
- ×生産国フィールドは直接呼べない



■静的のフィールドを呼ぶ場合

- ー日本車クラス.走るメソッド
- ー日本車クラス.生産国

∴クラスの設計書から直接実体を呼ぶ、単一のため値はクラス内で共通になる

1-3 静的なプロパティ

例題

「sample3.html」を参考に「example3.html」をやってみよう。

記述内容はコメントで指示しているので、それにしたがって記入する。

1-4 クラス継承

継承とは？

親クラスの情報を引き継ぐ= 継承する

設計書としてクラスを定義するが、似ている別物などがあったらどうするか。

- コピペして使う…
- 別物として新しく作成する…

そのような効率の悪さを解決するための機能が継承。

継承の具体的な動作とは？

親クラスのプロパティを利用することができる。

つまり、継承先の子クラスでは親クラスで不足したものを追加したり、上書きしたりして設計をし直すのが役割。

「sample4.html」を参照。

【記入例】

```
class childClass extends parentClass () {}
```

継承を図解してみよう

■Animalクラス

- constructor
- name(フィールド): 名前
- sound(メソッド): 鳴き声

fishクラスの「swim」は呼べない

仕様上メソッドについては、Javascriptのプロトタイプ継承というものが行われる。

継承

■Catクラス

- super(親コンストラクタ)
- constructor
- (継)name(フィールド): 名前
- (継)sound(メソッド): 鳴き声

fishクラスの「swim」は呼べない

■Dogクラス

- super(親コンストラクタ)
- constructor
- (継)name(フィールド): 名前
- (再定義)sound(メソッド): 鳴き声

fishクラスの「swim」は呼べない
親クラスの「sound」は呼べない

■Fishクラス

- super(親コンストラクタ)
- constructor
- (継)name(フィールド): 名前
- (継)sound(メソッド): 鳴き声
- swim(メソッド): 泳ぐ

1-4 コードを実装してみよう

演習

演習を通してクラスの記述に慣れよう。

最低限の情報とコメントを頼りに、コードを実装してみよう。
演習は複数あるので、「example-4-連番」になっている。

1-5 コードを実装してみよう

演習

単元のまとめとして「example5.html」の演習をしてみよう

最低限の情報とコメントを頼りに、コードを実装してみよう。

↓ コンソールの例

認証しました

トヨタのヤリスが走る。

トヨタのヤリスを運転する。

リチウムイオン搭載の日産のエリアが走る。

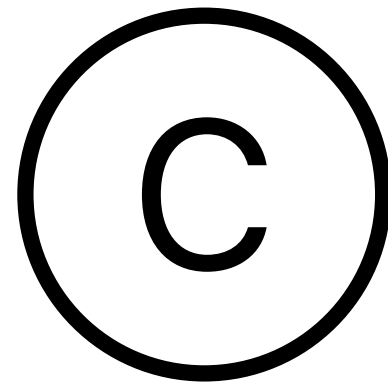
日産のエリアを運転する。

いすずのエルフが走る。

いすずのエルフを運転する。

積荷は食品です。

今回のまとめ



クラスの役割を理解する

ライブラリなどはクラスで定義されることは珍しくない。より複雑なモジュール処理を理解するためにも、クラスの基本的な使い方を理解しよう。



JavaScript

【改変】 実習 ファイルを分割して読み込んでみよう

今回の内容 - 教科書準拠

1.2-1 import/exportでファイルを分割しよう

2-1 import/exportでファイルを分割しよう

import/export

ファイルの分割と読み込みを実現する

外部ファイルに公開する場合は「export」、取り込む場合は「import」で宣言する。

✔ module機能を使うとセキュリティーモードになり、ローカルサーバー(LiveServer)を立てる必要がある。

- HTMLから読み込むときは、scriptの「type」を「module」にする
- スcopeが分かれるため、importした場所でロジックを書く

ファイル分割のメリットとは？

- 外部ライブラリなどを使うとき、ファイルをいくつも読み込む必要がない
- node.jsを始めとする開発環境を使う場合と考え方が似ている
 - →今後の講義の下地になるということ

2-1 import/exportでファイルを分割しよう

演習

サンプルや演習を通じて定着しよう

- 新しい構文を以下から確認しよう「late/chapter2/sup1/index.html」
- サンプルを確認してみよう「late/chapter2/sample1/index.html」
- 演習をしてみよう「late/chapter2/example1/index.html」



JavaScript

【改変】 実習 スプレッド構文およびレスト構文とAPI

今回の内容

- 1.3-1 スプレッド構文を使ってみよう
- 2.3-2 レスト構文を使ってみよう
- 3.3-3 APIの概要を捉えよう(説明のみ、次回実装)

3-1 スプレッド構文を使ってみよう

スプレッド構文とは？

スプレッド構文について

配列やオブジェクトを**展開**するための構文。
ドット3つ「...」を使って記述する。
コピーや結合などの用途で使われる。

サンプルコードを見よう。

配列やオブジェクトのコピーや結合のサンプル「<late/chapter3/sample1/index.html>」を実行してみよう

例) 結合の場合

```
const fruits = ['apple', 'banana'];  
const moreFruits = ['orange', ...fruits, 'grape'];
```

```
console.log(moreFruits); // 結果: ['orange', 'apple', 'banana', 'grape']
```

3-2 レスト構文を使ってみよう

レスト構文とは？

スプレッド構文について

配列やオブジェクトを**まとめる**ための構文(スプレッド構文とは逆)。
記述方法もスプレッド構文(送り手、代入側)と同じで、ドット3つ「...」を使って記述する(引数、受け手側)。
分割などの用途で使われる。

サンプルコードを見よう。

配列やオブジェクトの分割のサンプル「<late/chapter3/sample2/index.html>」を実行してみよう

例) 分割の場合

```
const numbers = [1, 2, 3, 4, 5];  
const [first, second, ...rest] = numbers;
```

```
console.log(first); // 1  
console.log(second); // 2  
console.log(rest); // [3, 4, 5]
```

3-2 レスト構文を使ってみよう

スプレッド/レスト構文の演習

演習を解いて理解を深めよう

「<late/chapter3/example2/index.html>」の問題を解いてみよう。

3-3 APIの概要を捉えよう(説明のみ、次回実装)

JavaScriptにおけるAPIとは？

APIとは？

API（Application Programming Interface）は、アプリケーション同士が情報をやり取りするための仕組み。
JavaScriptは外部との通信やクライアントレンダリングなどの仕組みをよく使うため、APIを使う場面は数多く存在する。

APIの使用例

- 外部サービスからデータを取得
 - 天気情報
 - ブログやサイト情報
 - マップ情報
 - クライアントレンダリングのビュー情報
- データの送信
 - フォーム処理
 - ログイン情報
 - ブログやマップの更新



JavaScript

実習 外部データと連携してみよう - 非同期処理を使ってみよう

今回の内容

- 1.4-1 同期処理と非同期処理
- 2.4-2 asyncとawait
- 3.4-3 外部APIを使った処理を実装してみよう

4-1 同期処理と非同期処理

同期処理とは？

同期処理のイメージを捉えよう

これまで実行してきたコードは、ほとんどは同期処理。
前の処理の完了を待って、順番に次の処理を実行する。
レジに並んで買い物をするようなイメージ。



歯磨き



食事



外出

左の作業が終わってから順番に右の作業に移る…同期処理

4-1 同期処理と非同期処理

非同期処理とは？

非同期処理のイメージを捉えよう

タイマー処理など、時間のかかる処理を分けて実行する方式。
いわゆる「マルチタスク」と言われる作業のことで、現実でも洗濯機や炊飯など日常生活ではよくある形。
JavaScriptはシングルスレッドで動作するため、擬似的なマルチスレッドとなる。



洗濯



炊飯



掃除

左の作業を待たず終わったものから作業に移る…非同期処理

✓現実では「音」で知らせてくれるが、プログラムでは完了後に実行する関数(コールバック)を設定する場合が多い

asyncとawaitを理解しよう

async, awaitとは？

非同期処理を同期処理に近い形で記述できるようにした構文。

非同期処理の宣言をasyncで行い、awaitで待つことができる。

旧来はPromiseのメソッドをチェーンで繋いで、成功処理を順々に書く (then)をとっていたが現在はネストが浅くシンプルに記述ができるようになった

asyncの役割

非同期処理の宣言を行う。

単体としての機能は特別なく、async側の記述は通常と同じように書ける。

awaitの役割

async関数の中でのみ記述することが可能。

※モジュール形式の場合はasync外のトップレベルで記述ができる(ES2022)。

直訳通り「待つ」「待機する」意味となり、非同期処理の完了を待って動作する。

✅ 非同期処理の場合は結果を待たないと処理中となるため、「結果が出たら実行する」というような使い方になる。

サンプルと演習をやってみよう「late/chapter4/」

4-3 外部APIを使った処理を実装してみよう

APIとは？

APIの実用例

API (Application Programming Interface) は「他のサービスと会話するためのルール」

例：天気予報アプリが気象庁のデータを取得する → 気象庁のAPIを使って通信

- 構造：リクエスト（要求） → サーバー → レスポンス（応答）
- 形式：多くは JSON で返ってくる
- HTTPメソッド(CRUDシステムでよく使われる)：
 - GET: データ取得、検索機能など
 - POST: データ送信(PUTの代替をする場合もあり)
 - PUT: データ更新
 - DELETE: データ削除

asyncとAPIを組み合わせて、サンプルと演習を解いてみよう。

「late/chapter4/」



JavaScript

実習 npmパッケージを使ってみよう

今回の内容

- 1.5-0 今回扱うソフトウェアの全体像を捉えよう
- 2.5-1 Homebrewをインストールしよう
- 3.5-2 Voltaをインストールしよう
- 4.5-3 Volta経由でnode.jsとnpmを使ってみよう
- 5.5-4 node.jsのサンプルプログラムを動かしてみよう

5-0 今回扱うソフトウェアの全体像を捉えよう

それぞれの名称と役割

完了目標 - JavaScript実行環境「node.js」を設定し、プログラム開発の効率化ができるようになる

- node.js
 - JavaScriptを動かすためのクロスプラットフォーム実行環境
 - node.jsのパッケージ(機能のプログラム群)を管理するツールとしてnpmを提供する
- npm(node package manager)
 - node.jsで実行されるパッケージ管理ソフト
 - プロジェクト毎にインストール済みのパッケージをJSONで管理することで、他の端末上でも実行環境を再現できるようにしている
- Volta
 - node.jsとnpmなどのパッケージ管理ソフトの複数インストールと切り替え、プロジェクトとバージョンを固定する機能を持つ
 - node.jsを端末本体にインストールした場合に過去のプロジェクトなどが動かなくなることがあるため、それを紐づけるためのソフトの1つ
- Homebrew
 - Mac/Linuxなどでパッケージを管理するソフト(npmはnode専用、Homebrewは本体端末の管理用)
 - 手動でインストールした場合、「何を」「どこに」「どのバージョンで」などの情報を自己管理する必要がある
 - Adobeソフトでいう「Creative Cloud」の立ち位置(一元管理の意味で)
- ※vite(node.js上で動作するビルドツール、今回はサンプルで使うだけ)

5-0 今回扱うソフトウェアの全体像を捉えよう

目標から作業を確認しよう

完了目標 - JavaScript実行環境「node.js」を設定し、プログラム開発の効率化ができるようになる

node.jsはJavaScriptのクロスプラットフォーム(OSなどに依存しない)で動作する実行環境。

クライアントサイドとして実行させる場合と、サーバーサイドで動作させる場合と両方対応ができる。

node.js自体が実行環境となるため、node.js上で動作するプログラム(パッケージ)のバージョンと整合性をとらなければならない。

→本体に直接node.jsを入れずに、バージョン切り替えツール「Volta」を通じて簡単にイン/アンインストール、切り替え可能な状態にする。

Macのパッケージ管理: Homebrew

Node.jsのバージョン管理と切り替え: Volta

JavaScript実行環境: node.js

node.jsのパッケージ管理: npm

- パッケージA
 - 依存A-1
 - 依存A-2
 - 依存A-3
- パッケージB
- パッケージC

5-2 Voltaをインストールしよう

[Homebrew]概要とインストール

Homebrewとは？

主にMacOS(Linuxも可)で使われるパッケージ管理ソフト。
パッケージの一元管理、アップデート、削除などを行うことができる。
個別に入れるよりも後々楽になる可能性が高いので、Homebrew経由がおすすめ。

Homebrewのインストール方法

公式にある通りコマンドからインストールする。
※今回のコマンドは全て、「[introduction/node/node-init.md/pdf](https://brew.sh/ja/introduction/node/node-init.md/pdf)」に入れている
<https://brew.sh/ja/>

5-2 Voltaをインストールしよう

[Volta]概要とインストール

Voltaとは？

Mac/Windows/Linuxとクロスプラットフォームで動作するJavaScriptの「ツールマネージャー」。

- node.js/npm(類似のyarnなども可)を一元管理することができる
 - 複数バージョンのインストール
 - インストール済みバージョンの確認
 - プロジェクト(ディレクトリ)単位でのバージョン切り替え
 - gitなどを通じてファイルで共有が可能
 - グローバルバージョンの指定

Voltaのインストール方法

Homebrew経由でインストールする。

5-3 Volta経由でnode.jsとnpmを使ってみよう

[node.js]概要とインストール

node.jsとは？

Mac/Windows/Linux(サーバーのWebサーバー含む)とクロスプラットフォームで動作するJavaScriptの実行環境。

- クライアント側では、強力なツールを使って開発を効率化し実行可能な形式に変換するような使い方が可能
 - pug → HTML変換
 - sass → css変換
 - TypeScript → JavaScript変換
- サーバー側では、ウェブサーバーや中継として使用できる
 - →AWSのlambda

npmとは？

node.js上で動作するパッケージ管理マネージャー。

また、node.jsプログラムを実行する機能を持つ。

✅類似ソフトとして「yarn」、「pnpm」などがある。

node.js/npmのインストール方法

Volta経由でインストールする。

5-4 node.jsのサンプルプログラムを動かしてみよう

nodeでJavaScriptを実行しよう

npmでプログラムを実行しよう

node.jsのプログラムはnpmコマンド経由で実行することができる。

- package.json
 - npmでインストールしたパッケージの管理
 - nodeスクリプトも同JSONに記述する(実行するjsファイルを指定する)
-  package-lock.json
 - それぞれのパッケージがさらに依存しているパッケージ(副依存)まで管理するファイル
 - (イメージ)原材料だけでなく、購入先や品物の型番なども全て記録する

※vite

クライアント側のバンドルとして今回は使用する。
vue.jsを作った方が開発したもの。